

02 - Supervised Learning

Alessandro Bregoli

2025

Introduction

In this tutorial we will **describe** how to manage a **classification task** using **Knime**.



Introduction

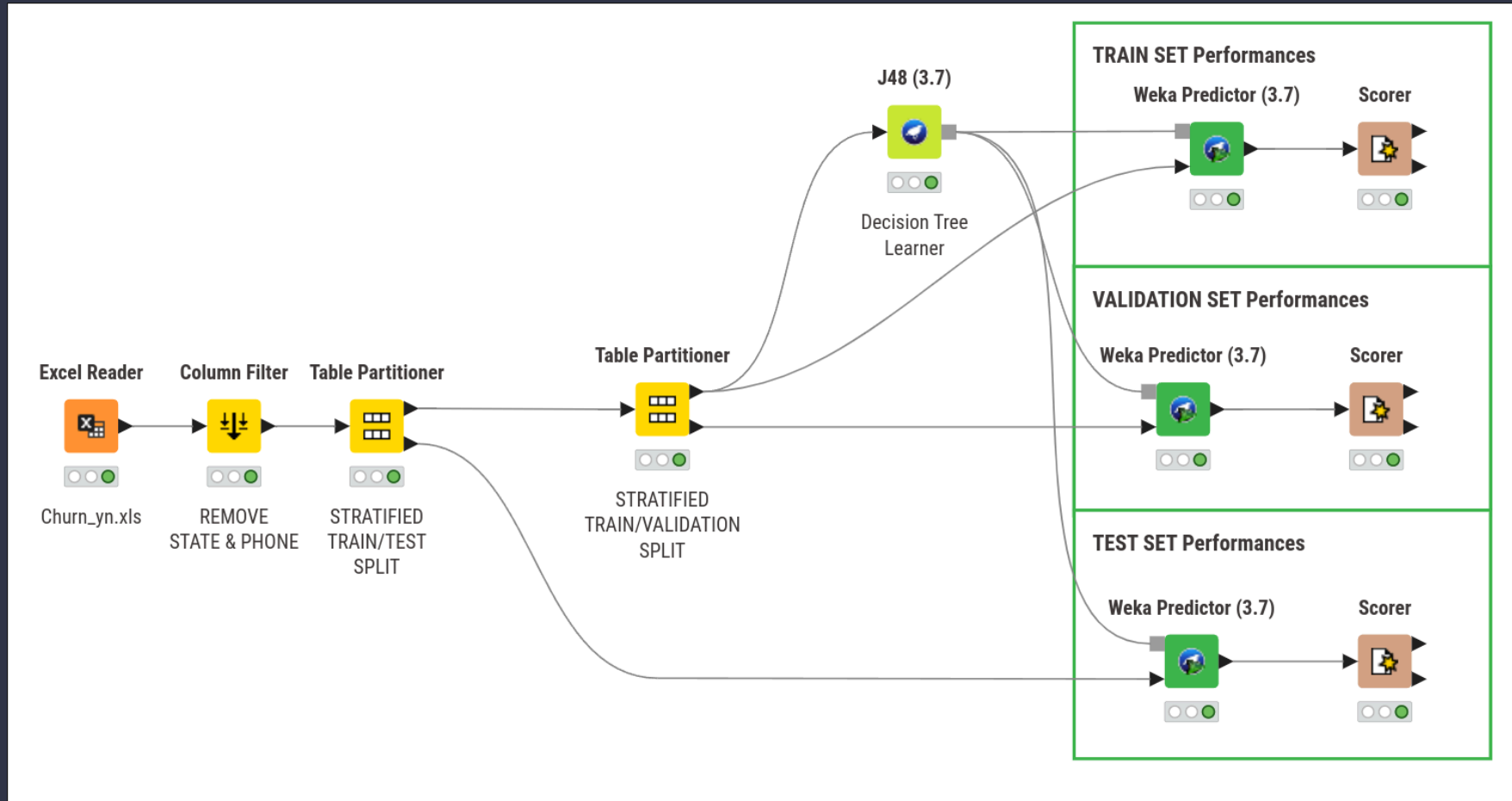
In this tutorial we will **describe** how to manage a **classification task** using **Knime**.

Furthermore, we will introduces the most common classification algorithms:

- Decision Tree
- Random Forest
- Gradient Boosted Trees
- Logistic Regression
- Support Vector Machine
- Multilayer Perceptron
- Naive Bayes
- Tree Augmented Naive Bayes
- Bayesian Network



Example



Validation - Confusion Matrix

The **evaluation** of a **classification model** must quantify how well the model is able to **predict the right class**.

Validation - Confusion Matrix

The **evaluation** of a **classification model** must quantify how well the model is able to **predict the right class**.

For a **binary classifier**, the **Confusion Matrix** counts the number of correct and wrong prediction as follows:

		Predicted Class	
		False	True
Actual Class	False	TN	FP
	True	FN	TP

- **TP**: True Positive
- **TN**: True Negative
- **FP**: False Positive
- **FN**: False Negative

Validation - Scorer

The *validation* node compute the **confusion matrix** (first output) and a set of validation metrics (second output) such as:

-
-
-
-

Scorer



Validation - Scorer

The *validation* node compute the **confusion matrix** (first output) and a set of validation metrics (second output) such as:

- **Precision**

-
-
-

Precision measures the proportion of **positive predictions** that were **actually correct**.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Scorer



Validation - Scorer

The *validation* node compute the **confusion matrix** (first output) and a set of validation metrics (second output) such as:

- Precision
- **Recall**
-
-

Recall measures the **proportion of actual positive cases** that were **correctly identified**.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Scorer



Validation - Scorer

The *validation* node compute the **confusion matrix** (first output) and a set of validation metrics (second output) such as:

- Precision
- Recall
- **F1**
-

F1 is the **harmonic mean** of **precision** and **recall**.

$$F1 = \frac{2}{\text{Precision}^{-1} + \text{Recall}^{-1}}$$

Scorer



Validation - Scorer

The *validation* node compute the **confusion matrix** (first output) and a set of validation metrics (second output) such as:

- Precision
- Recall
- F1
- **Accuracy**

Accuracy measures the **proportion** of cases **correctly classified**.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Scorer



Weka

Weka is a collection of machine learning algorithms for data mining tasks. It contains tools for data preparation, classification, regression, clustering, association rules mining, and visualization.

— Weka Website

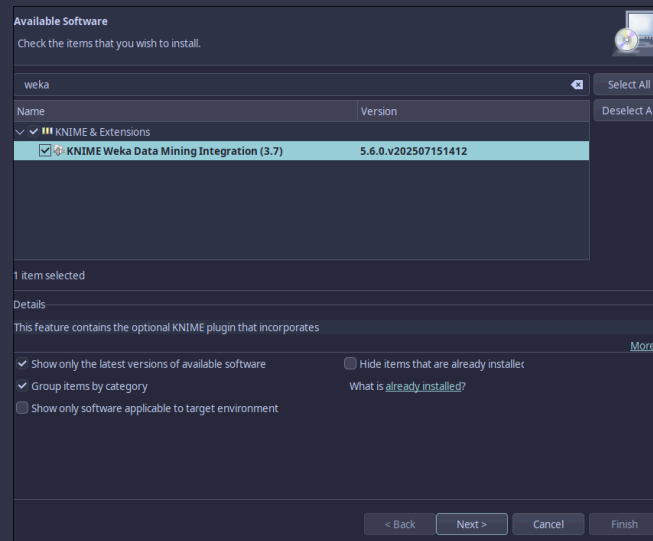


Weka

Weka is a collection of machine learning algorithms for data mining tasks. It contains tools for data preparation, classification, regression, clustering, association rules mining, and visualization.

— Weka Website

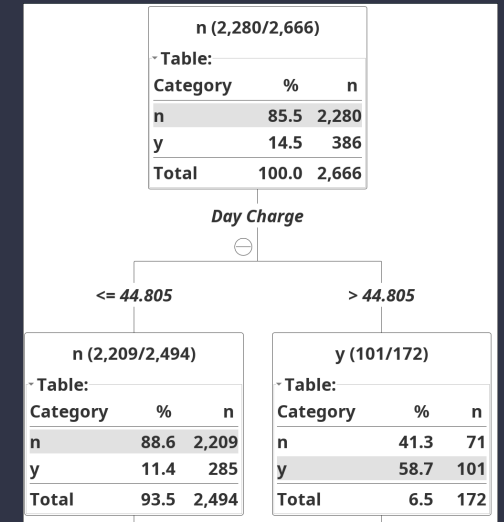
Menu->Install Extensions



Decision Tree

Decision Tree

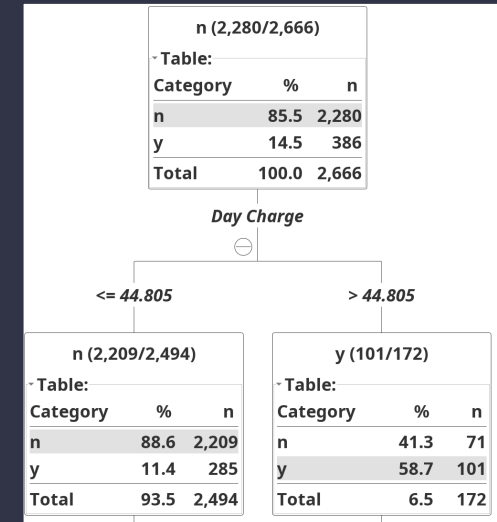
A *decision tree* classifier, consists in a set of **consecutive rules** applied to predict the class of the instance under analysis.



Decision Tree

A *decision tree* classifier, consists in a set of **consecutive rules** applied to predict the class of the instance under analysis.

Each **node** identify a specific **rule**; while the **edges** between two nodes identify the order in which the rule must be applied.

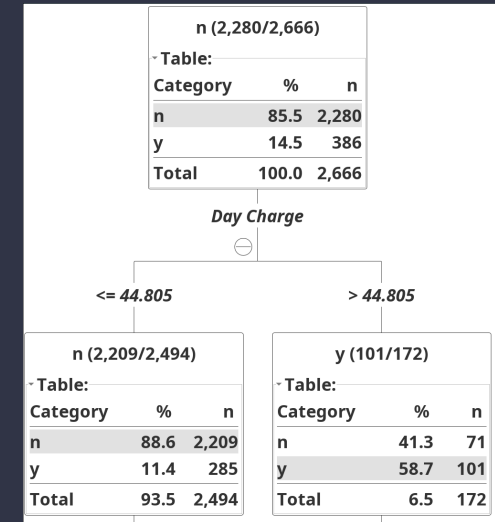


Decision Tree

A *decision tree* classifier, consists in a set of **consecutive rules** applied to predict the class of the instance under analysis.

Each **node** identify a specific **rule**; while the **edges** between two nodes identify the order in which the rule must be applied.

The sequence of decision start from the **root** node (top node) and ends in one of the **leaf** nodes (bottom nodes)



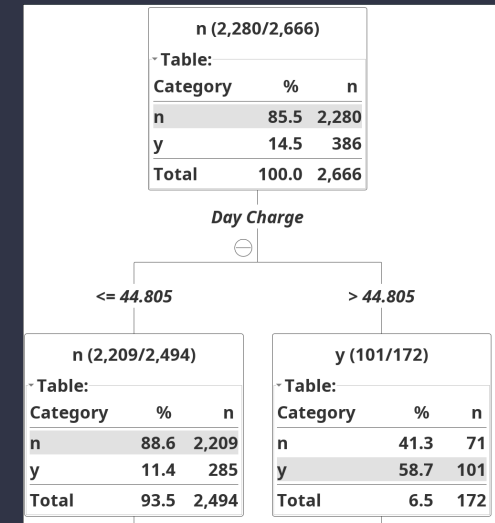
Decision Tree

A *decision tree* classifier, consists in a set of **consecutive rules** applied to predict the class of the instance under analysis.

Each **node** identify a specific **rule**; while the **edges** between two nodes identify the order in which the rule must be applied.

The sequence of decision start from the **root** node (top node) and ends in one of the **leaf** nodes (bottom nodes)

During the **learning phase**, the algorithm start identifying the **root node** and than, it proceeds **top-down**.



Decision Tree

A *decision tree* classifier, consists in a set of **consecutive rules** applied to predict the class of the instance under analysis.

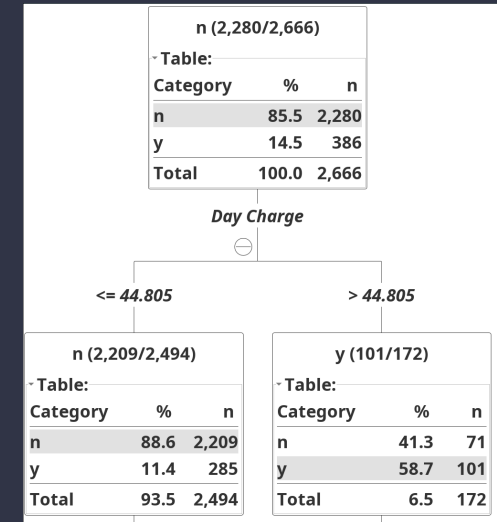
Each **node** identify a specific **rule**; while the **edges** between two nodes identify the order in which the rule must be applied.

The sequence of decision start from the **root** node (top node) and ends in one of the **leaf** nodes (bottom nodes)

During the **learning phase**, the algorithm start identifying the **root node** and than, it proceeds **top-down**.

At each step, the best discriminating feature is selected accordingly to a metric such as:

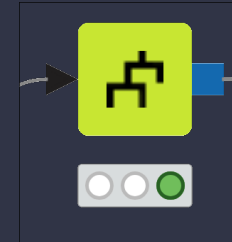
- Gini Impurity
- Information Gain
- Variance Reduction



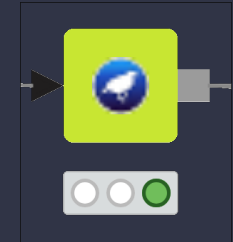
Decision Tree Nodes

There are multiple nodes implementing *decision trees*.

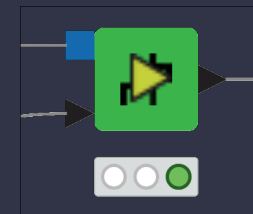
Decision Tree
Learner



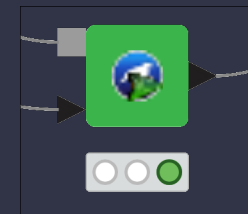
J48



Decision Tree
Predictor



Weka
Predictor

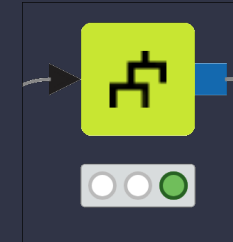


Decision Tree Nodes

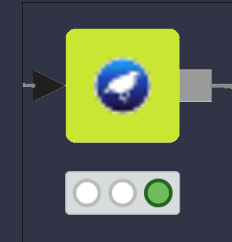
There are multiple nodes implementing *decision trees*.

The *Decision Tree Learner* and *Decision Tree Predictor* nodes are part of the core Knime node library and implements a **naïve** version of the **decision tree learning algorithm**.

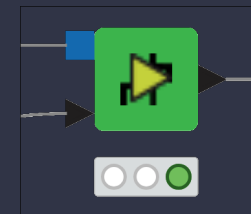
Decision Tree
Learner



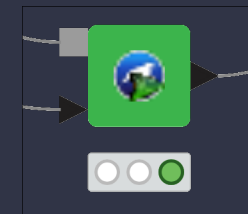
J48



Decision Tree
Predictor



Weka
Predictor



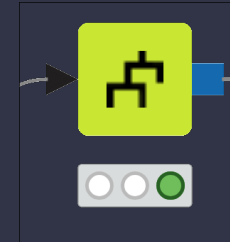
Decision Tree Nodes

There are multiple nodes implementing *decision trees*.

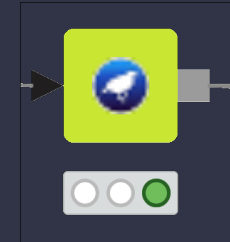
The *Decision Tree Learner* and *Decision Tree Predictor* nodes are part of the core Knime node library and implements a **naïve** version of the **decision tree learning algorithm**.

The *J48* and *Weka Predictor* nodes are provided by **Weka extension** and implement the **C4.5**, a slightly more sophisticated **decision tree learning algorithm**.

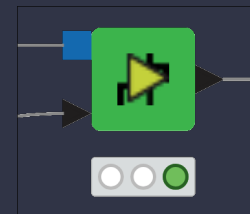
Decision Tree
Learner



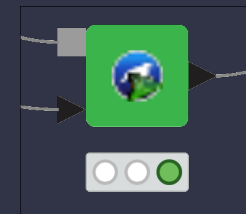
J48



Decision Tree
Predictor



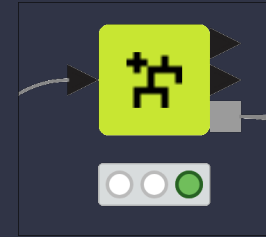
Weka
Predictor



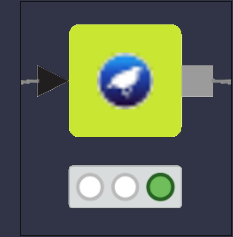
Random Forest Nodes

The *random forest* is an **ensemble learning** algorithm that combines **multiple decision trees**.

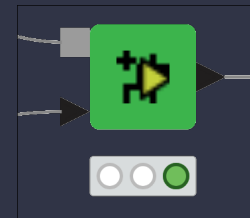
Random Forest
Learner



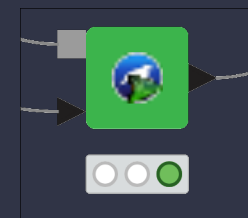
RandomForest



Decision Tree
Predictor



Weka
Predictor

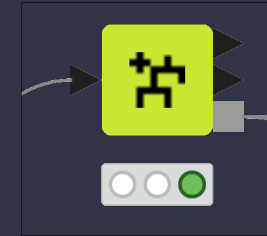


Random Forest Nodes

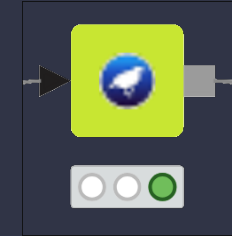
The *random forest* is an **ensemble learning** algorithm that combines **multiple decision trees**.

The core idea is to **learn multiple decision trees** over **subset of features** and **instances**

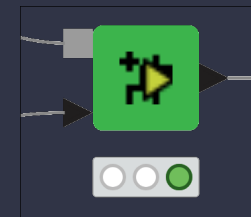
Random Forest
Learner



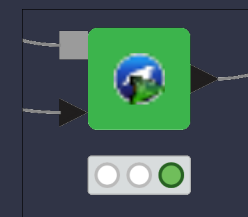
RandomForest



Decision Tree
Predictor



Weka
Predictor



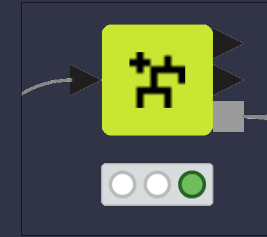
Random Forest Nodes

The *random forest* is an **ensemble learning** algorithm that combines **multiple decision trees**.

The core idea is to **learn multiple decision trees** over **subset of features** and **instances**

The **prediction** is performed aggregating the prediction of the decision trees by **majority vote**

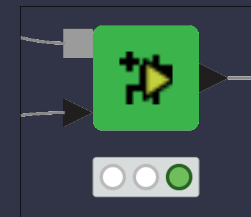
Random Forest
Learner



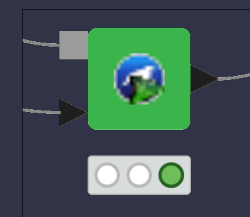
RandomForest



Decision Tree
Predictor



Weka
Predictor



Random Forest Nodes

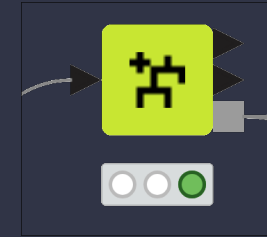
The *random forest* is an **ensemble learning** algorithm that combines **multiple decision trees**.

The core idea is to **learn multiple decision trees** over **subset of features** and **instances**

The **prediction** is performed aggregating the prediction of the decision trees by **majority vote**

Also for the **random forests** there are the knime implementation (*Random Forest Learner* node - *Random Forest Predictor* node) and the weka implementation (*RandomForest* node - *Weka Predictor*).

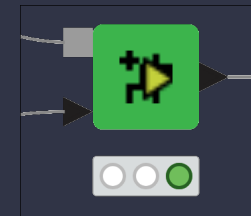
Random Forest
Learner



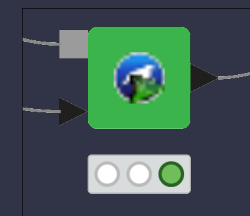
RandomForest



Decision Tree
Predictor



Weka
Predictor



Gradient Boosted Tree

Gradient Boosting is an ensemble method that **combines weak learners** to create a single **strong learner**.

Gradient Boosted Tree

Gradient Boosting is an ensemble method that **combines weak learners** to create a single **strong learner**. The *Gradient Boosted Tree* algorithm is specific kind of **gradient boosting** that **sequentially** combines **decision trees**.

Gradient Boosted Tree

Gradient Boosting is an ensemble method that **combines weak learners** to create a single **strong learner**. The *Gradient Boosted Tree* algorithm is specific kind of **gradient boosting** that **sequentially** combines **decision trees**.

Each new **tree** is specifically trained to **fix** the **wrong predictions** of all the **previous trees**.

Gradient Boosted Tree

Gradient Boosting is an ensemble method that **combines weak learners** to create a single **strong learner**. The *Gradient Boosted Tree* algorithm is specific kind of **gradient boosting** that **sequentially** combines **decision trees**.

Each new **tree** is specifically trained to **fix** the **wrong predictions** of all the **previous trees**.

Comparison of **Random Forest** and **Gradient Boosted Tree**

Characteristic	Random Forest	Gradient Bosted Tree
Enseble method	Baggin	Boosting
Training	Idependent trees	Sequential trees

Gradient Boosted Tree

Gradient Boosting is an ensemble method that **combines weak learners** to create a single **strong learner**. The *Gradient Boosted Tree* algorithm is specific kind of **gradient boosting** that **sequentially** combines **decision trees**.

Each new **tree** is specifically trained to **fix the wrong predictions** of all the **previous trees**.

Comparison of Random Forest and Gradient Boosted Tree

Characteristic	Random Forest	Gradient Bosted Tree
Enseble method	Baggin	Boosting
Training	Idependent trees	Sequential trees

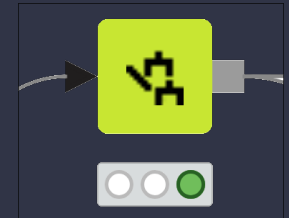
There are many implementation of Gradient Boosted Trees:

- Knime Gradient Boosted Tree
- XGBOOST
- Weka MultiBoost
- LightGBM

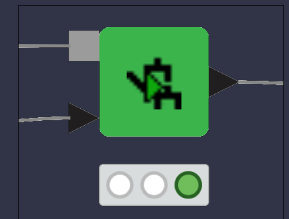
Gradient Boosted Tree Node

The *Gradient Boosted Tree Learner* and *Gradient Boosted Tree Predictor* nodes are part of the core knime node library.

**Gradient
Boosted Tree
Learner**



**Gradient
Boosted Tree
Predictor**

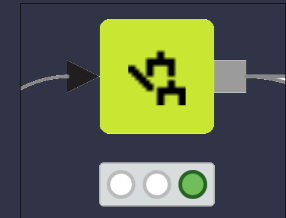


Gradient Boosted Tree Node

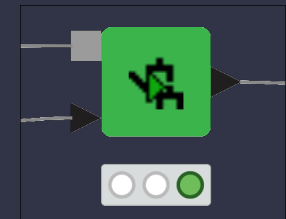
The *Gradient Boosted Tree Learner* and *Gradient Boosted Tree Predictor* nodes are part of the core knime node library.

The screenshot shows the configuration window for the Gradient Boosted Tree Learner node. It has four tabs: Options, Advanced Options, Flow Variables, and Job Manager Selection. The 'Options' tab is active. The 'Target Column' is set to 'Churn'. Under 'Attribute Selection', the 'Use column attributes' radio button is selected. The 'Manual Selection' radio button is also selected. The 'Exclude' list is empty, and the 'Include' list contains 'Account Length', 'VMail Message', 'Day Mins', 'Eve Mins', 'Night Mins', 'Intl Mins', 'CustServ Calls', and 'Intl Plan'. The 'Enforce exclusion' radio button is selected. The 'Tree Options' section has 'Limit number of levels (tree depth)' checked and set to 4. The 'Boosting Options' section has 'Number of models' set to 100 and 'Learning rate' set to 0.05.

**Gradient
Boosted Tree
Learner**



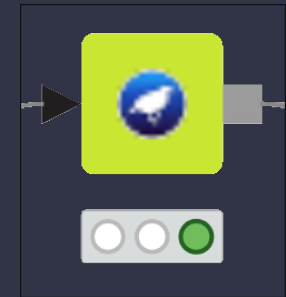
**Gradient
Boosted Tree
Predictor**



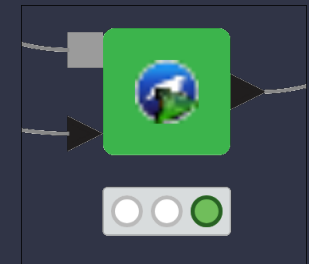
Gradient Boosted Tree Weka Node

The *MultiBoostAB* and *Weka Predictor* nodes are part of the Weka extension.

MultiBoostAB



Weka Predictor

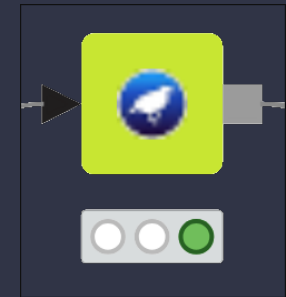


Gradient Boosted Tree Weka Node

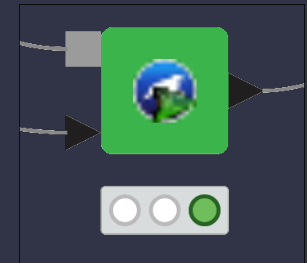
The *MultiBoostAB* and *Weka Predictor* nodes are part of the Weka extension.

The weka library implement a **generic Gradient Boosting** algorithm.

MultiBoostAB



Weka Predictor



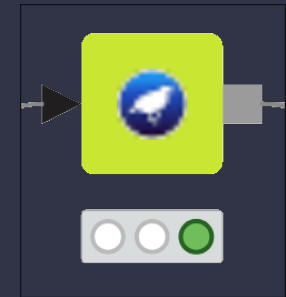
Gradient Boosted Tree Weka Node

The *MultiBoostAB* and *Weka Predictor* nodes are part of the Weka extension.

The weka library implement a **generic Gradient Boosting** algorithm.

In order to perform a **Gradient Booste Tree** we need to configure the *MultiBoostAB* node selecting a **decision tree** as **week classifier** such as: **J48** or **REPTree**.

MultiBoostAB



Weka Predictor



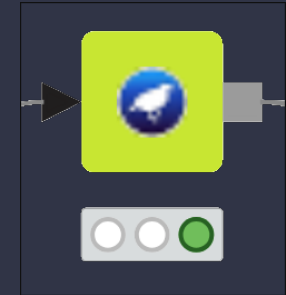
Gradient Boosted Tree Weka Node

The *MultiBoostAB* and *Weka Predictor* nodes are part of the Weka extension.

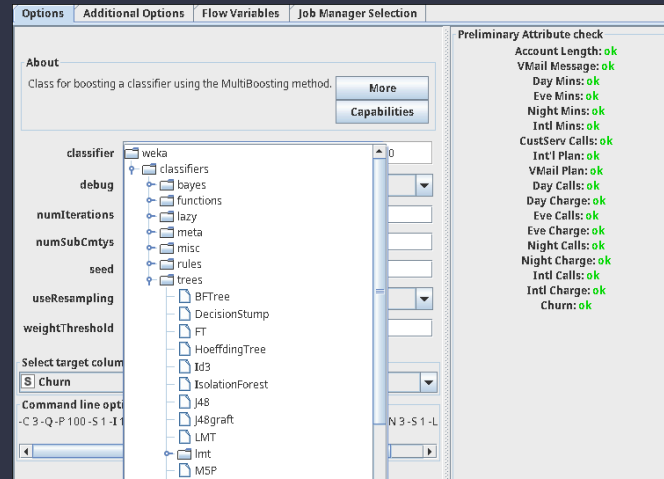
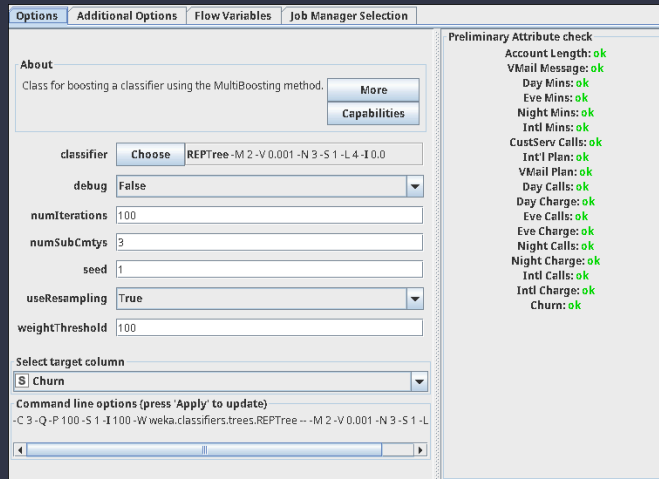
The weka library implement a **generic Gradient Boosting** algorithm.

In order to perform a **Gradient Booste Tree** we need to configure the *MultiBoostAB* node selecting a **decision tree** as **week classifier** such as **J48** or **REPTree**.

MultiBoostAB



Weka Predictor



Hands on (30 minutes)

- **Import** the **Churn_yn.xls**.
- **Filter** the columns: State, Area Code and Phone.
- **Split** the dataset in two partition (80% train, 20% test)
- **Train** the following classifiers using the train partition (80% of data):
 - Decision Tree
 - Gradient Boosted Tree
 - J48 (weka)
 - MultiBoostAB (weka)
 - Random Forest
 - *Hint: from the classifier option*
 - RandomForest (weka)
 - select J48 or REPTree*
- **Evaluate** the performance of the trained classifiers on both **train** and **test**.

BONUS

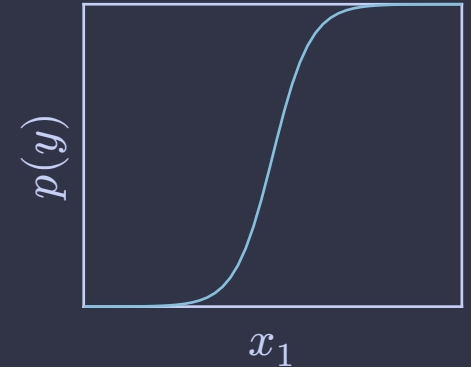
Install the knime extension for **XGBOOST** and evaluate the its performances.



Logistic Regression & Support Vector Machine

Logistic Regression

The *Logistic Regression* is a **linear classifier** that **models** the **probability** of observing a specific class as a **linear combination** of the **independent** variables.



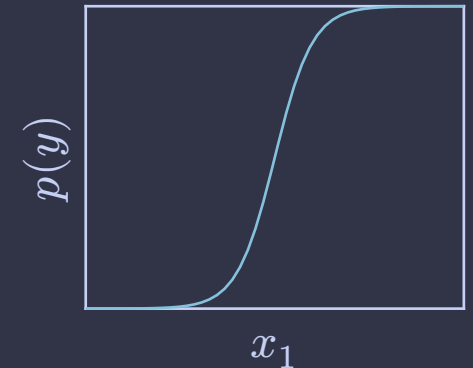
Logistic Regression

The *Logistic Regression* is a **linear classifier** that **models** the **probability** of observing a specific class as a **linear combination** of the **independent** variables.

$$p(y) = \frac{1}{1 + e^{-\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n}}$$

where:

- y is the predicted variable (i.e. *Churn* in churn_yn dataset)
- β_0 is the intercept of linear regression
- x_i are the independent variables (i.e. *Day Mins* in churn_yn dataset)
- β_i are the slopes of the linear regression



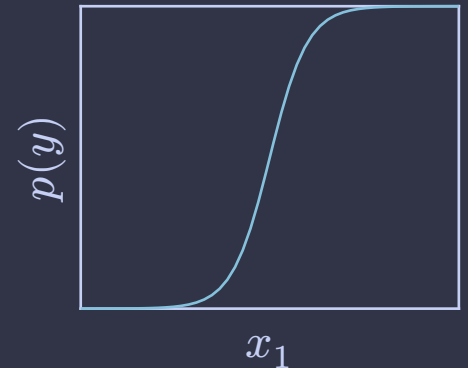
Logistic Regression

The *Logistic Regression* is a **linear classifier** that **models** the **probability** of observing a specific class as a **linear combination** of the **independent** variables.

$$p(y) = \frac{1}{1 + e^{-\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n}}$$

where:

- y is the predicted variable (i.e. *Churn* in churn_yn dataset)
- β_0 is the intercept of linear regression
- x_i are the independent variables (i.e. *Day Mins* in churn_yn dataset)
- β_i are the slopes of the linear regression

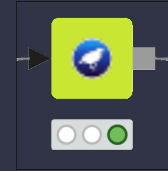


When using the **logistic regression** it is important to rescale the features with a **min-max scaler** or a **normalization** otherwise features with higher magnitude will be considered more important.

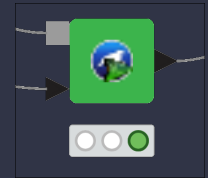
Logistic Regression Node

The *Logistic* and *Weka Predictor* nodes are part of the Weka extension.

Logistic



Weka Predictor

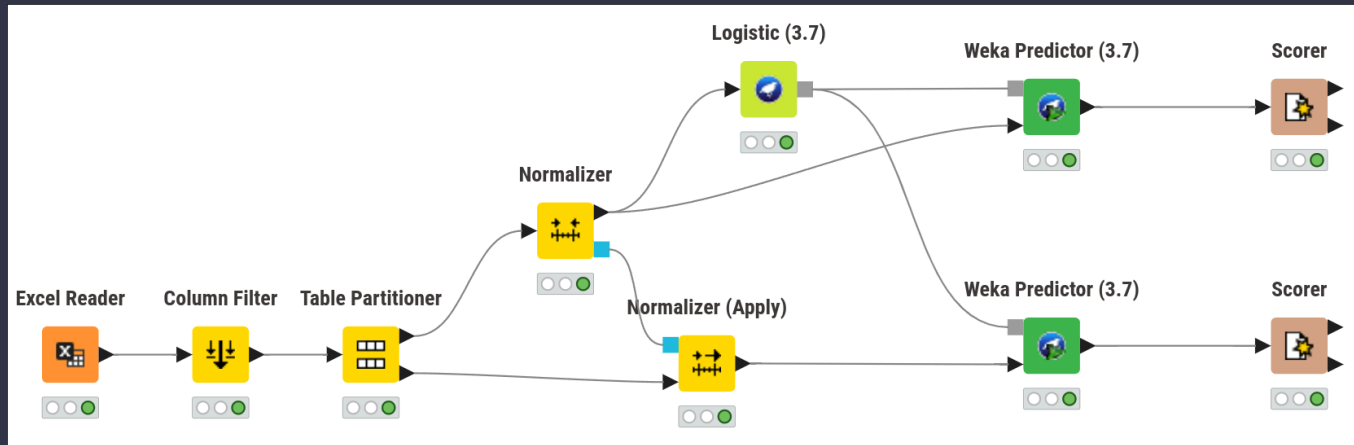


Normalizer

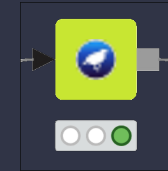


Logistic Regression Node

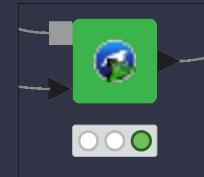
The *Logistic* and *Weka Predictor* nodes are part of the Weka extension.



Logistic



Weka Predictor

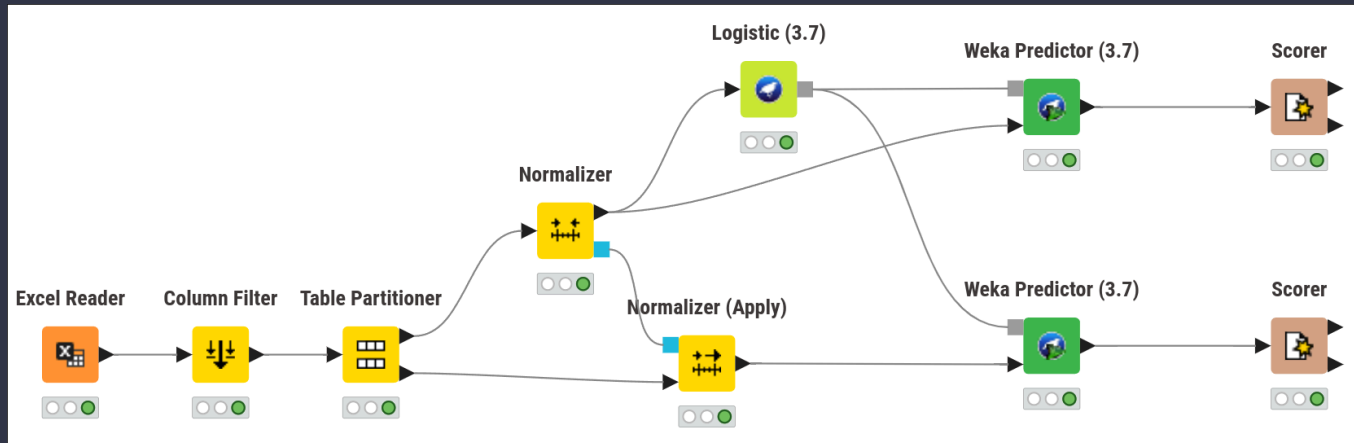


Normalizer

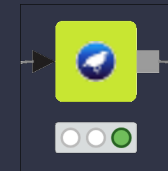


Logistic Regression Node

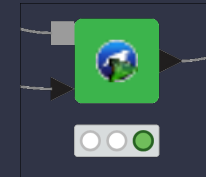
The *Logistic* and *Weka Predictor* nodes are part of the Weka extension.



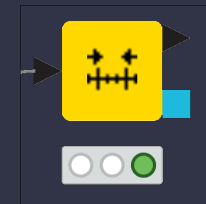
Logistic



Weka Predictor



Normalizer

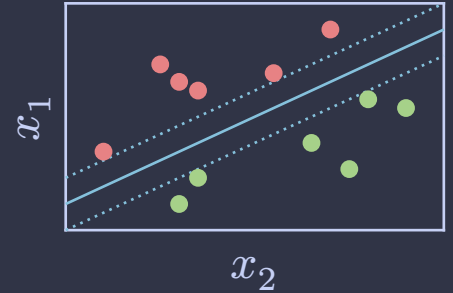


When evaluating a classifier it is important do not leak information from the test set to the train set.

- **Normalizer**: Calculate the normalization parameters and apply the transformation to the train set.
- **Normalizer (Apply)**: Apply the transformation to the test test using the parameters learned on the train set.

Support Vector Machine

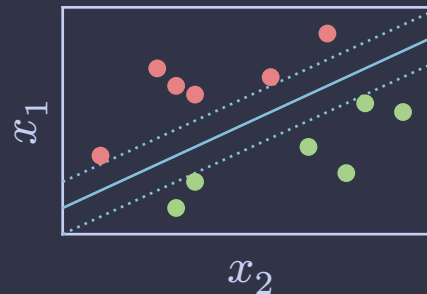
Support Vector Machine (SVM) is a linear classifier that aims to identify the wider separation region between the groups of elements.



Support Vector Machine

Support Vector Machine (SVM) is a linear classifier that aims to identify the wider separation region between the groups of elements.

The main **strength** of this approach is a lower risk of **overfitting**.

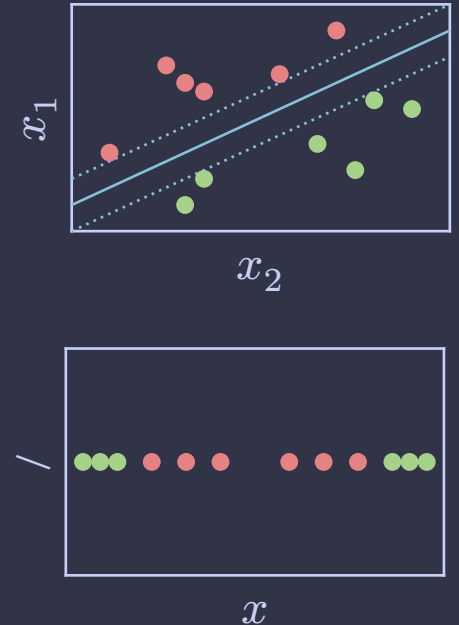


Support Vector Machine

Support Vector Machine (SVM) is a linear classifier that aims to identify the wider separation region between the groups of elements.

The main **strength** of this approach is a lower risk of **overfitting**.

However, the SVM can only identify **linearly separable classes**.



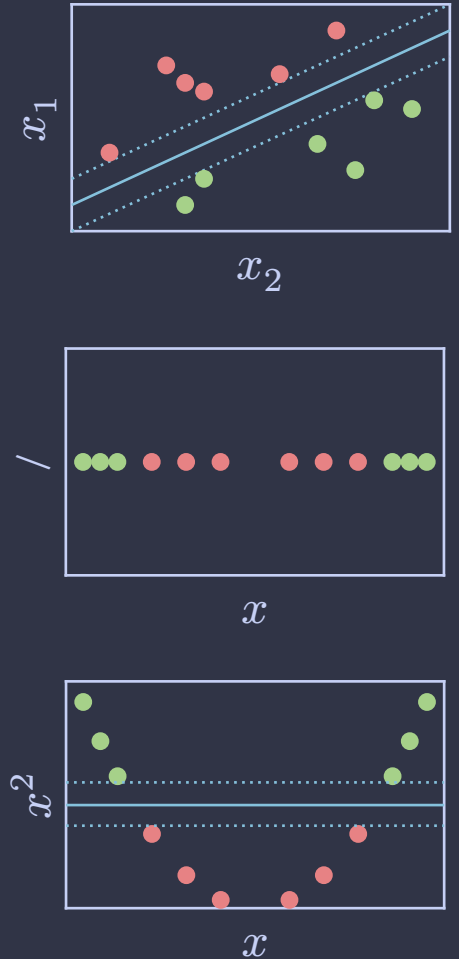
Support Vector Machine

Support Vector Machine (SVM) is a linear classifier that aims to identify the wider separation region between the groups of elements.

The main **strength** of this approach is a lower risk of **overfitting**.

However, the SVM can only identify **linearly separable classes**.

To overcome this limitation and classify **non-linearly separable** dataset the **kernel approach** can be applied.



Support Vector Machine

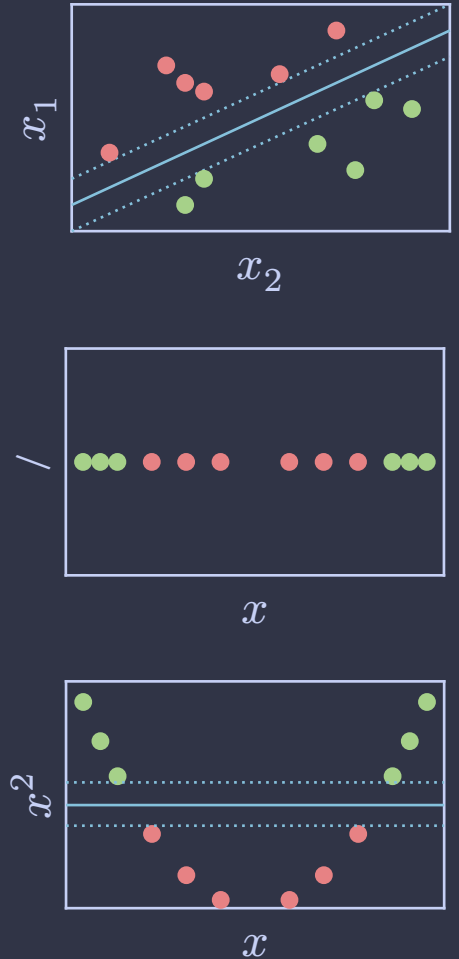
Support Vector Machine (SVM) is a linear classifier that aims to identify the wider separation region between the groups of elements.

The main **strength** of this approach is a lower risk of **overfitting**.

However, the SVM can only identify **linearly separable classes**.

To overcome this limitation and classify **non-linearly separable** dataset the **kernel approach** can be applied.

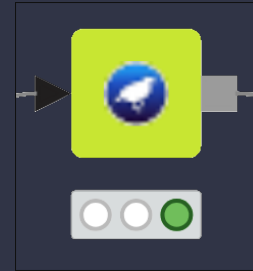
The idea is to **generate a feature space** where the classification problem can be solved with a linear classifier.



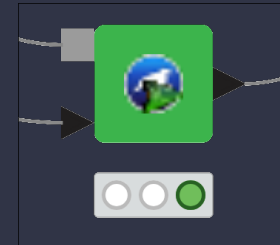
Support Vector Machine Node

The *SMO* is one of the Weka implementation of SVM.

SMO



Weka Predictor

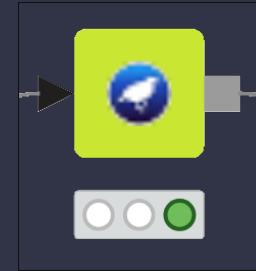


Support Vector Machine Node

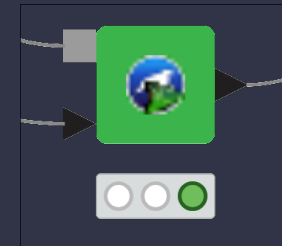
The *SMO* is one of the Weka implementation of SVM.

Similarly to Logistic Regression, also for SVM **normalization** is required. However, the SMO node take care of this problem internally.

SMO



Weka Predictor



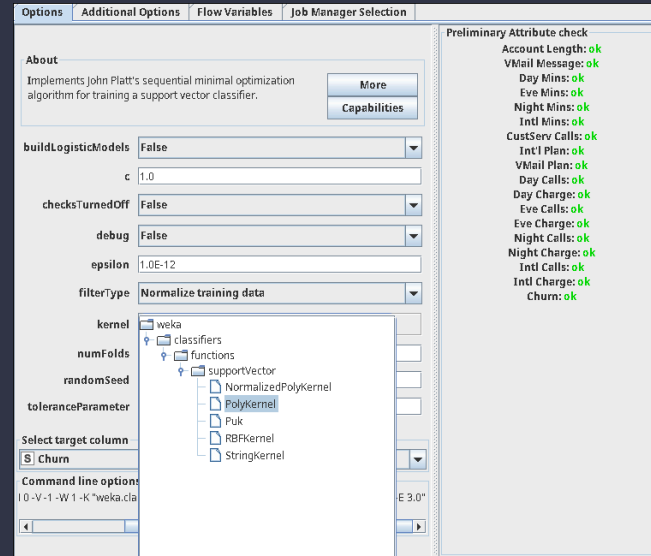
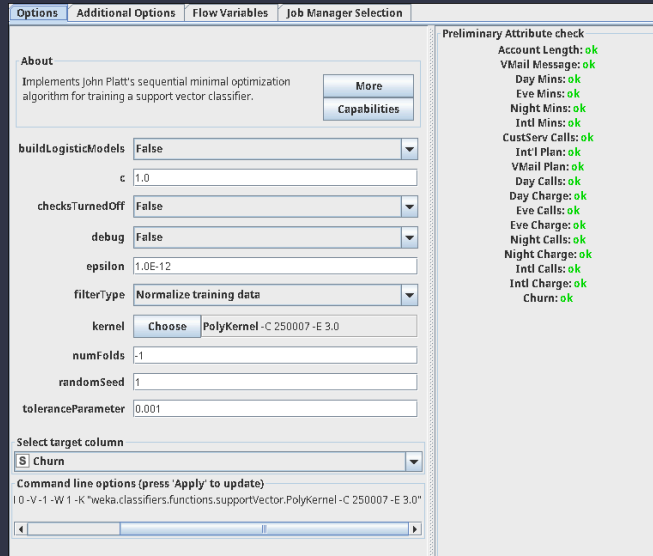
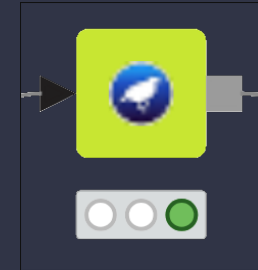
Support Vector Machine Node

The SMO is one of the Weka implementation of SVM.

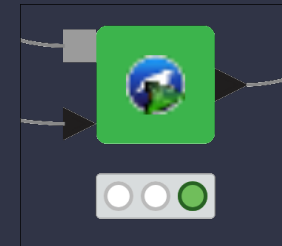
Similarly to Logistic Regression, also for SVM **normalization** is required. However, the SMO node take care of this problem internally.

Both **normalization** and **kernel selection** can be activated from the configuration panel:

SMO

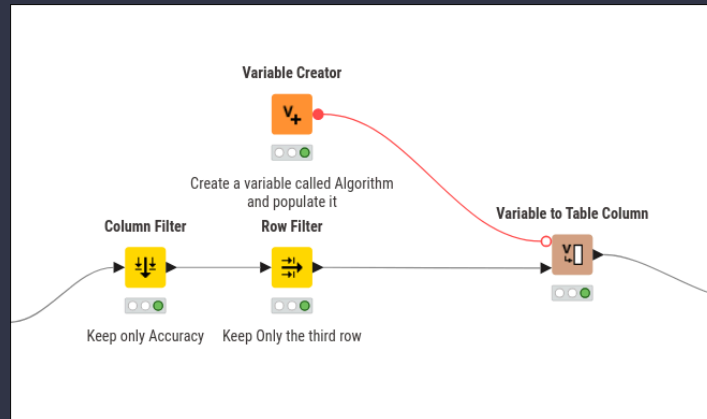


Weka Predictor



Hands on (30 minutes)

- **Import** the **Churn_yn** dataset.
- **Filter** the columns: State, Area Code and Phone
- **Split** the dataset in two partition (80% train, 20% test)
- **Train** a logistic regression and an SVM (SMO node)
- **Validate** both train and test with a **scorer**.
- For each algorithm extract the **accuracy**:

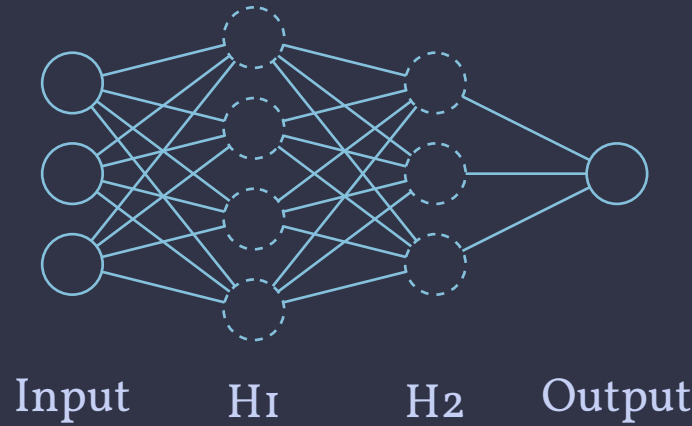


- Use the **Concatenate** node to combine all the results and the **Boxplot** node to show them.



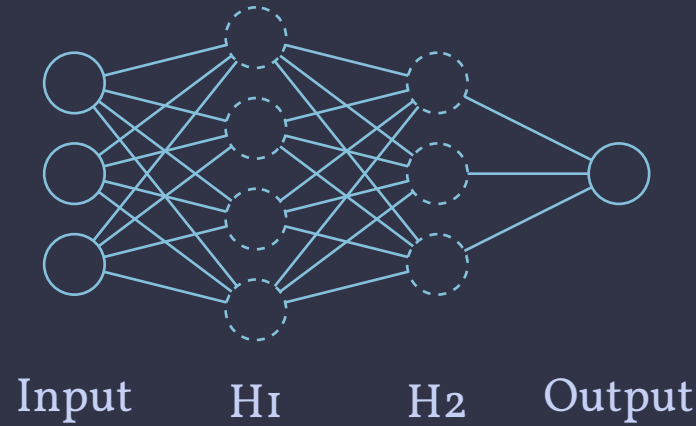
Multi Layer Perceptron

Multi-Layer Perceptron



The *Multi-Layer Perceptron* (MLP) is a **feedforward fully connected neural network**.

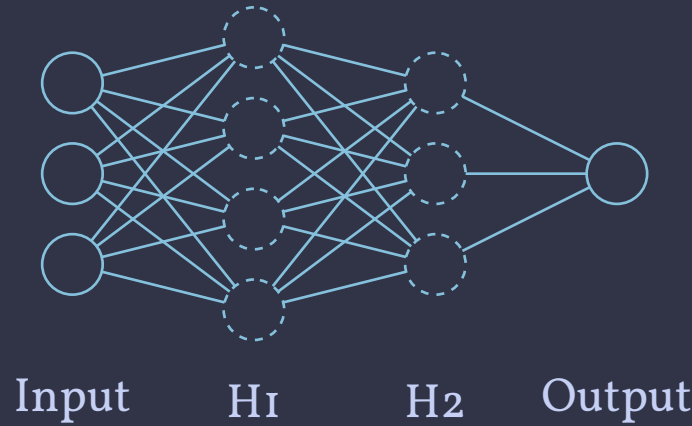
Multi-Layer Perceptron



The *Multi-Layer Perceptron* (MLP) is a **feedforward fully connected neural network**.

Neural Networks can have many architectures. However, in MLP we have that all the node in a layer are connected with all the node in the previous and in the next layer (fully connected).

Multi-Layer Perceptron

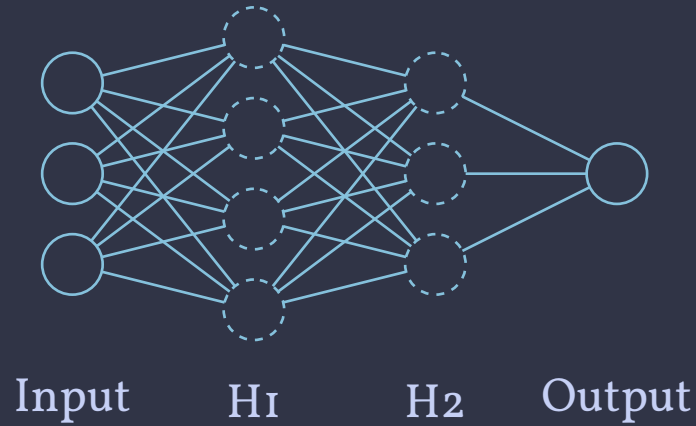


The *Multi-Layer Perceptron* (MLP) is a **feedforward fully connected neural network**.

Neural Networks can have many architectures. However, in MLP we have that all the node in a layer are connected with all the node in the previous and in the next layer (fully connected).

Furthermore, information flows from input to output (feedforward).

Multi-Layer Perceptron



The *Multi-Layer Perceptron* (MLP) is a **feedforward fully connected neural network**.

Neural Networks can have many architectures. However, in MLP we have that all the node in a layer are connected with all the node in the previous and in the next layer (fully connected).

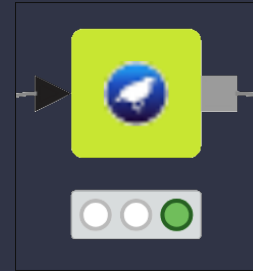
Furthermore, information flows from input to output (feedforward).

Thus, the only architectural decision for an MLP are the **number of hidden layers** and the **number of nodes** in a hidden layer.

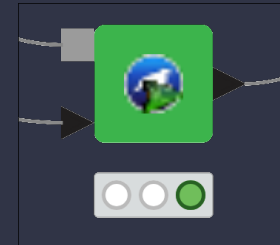
Multi Layer Perceptron Node

The *MultilayerPerceptron* node is a Weka implementation of MLP.

MultilayerPerceptron



Weka Predictor

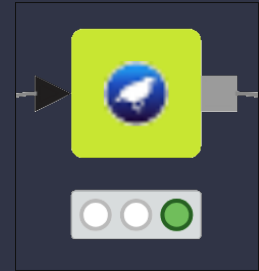


Multi Layer Perceptron Node

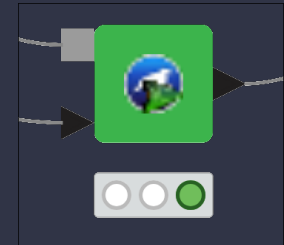
The *MultilayerPerceptron* node is a Weka implementation of MLP.

Similarly to Logistic Regression and SVM, also for MLP **normalization** is required. The MultilayerPerceptron node can perform the normalization setting the corresponding flag in the configuration panel.

MultilayerPerceptron



Weka Predictor



Multi Layer Perceptron Node

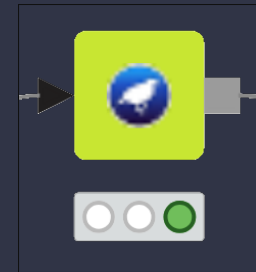
The *MultilayerPerceptron* node is a Weka implementation of MLP.

Similarly to Logistic Regression and SVM, also for MLP **normalization** is required. The MultilayerPerceptron node can perform the normalization setting the corresponding flag in the configuration panel.

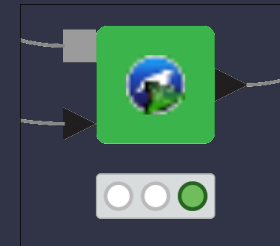
The most relevant parameters are:

- **hiddenLayer**: a sequence of comma separated integer representing the number of nodes for each hidden layer.
-
-

MultilayerPerceptron



Weka Predictor



Multi Layer Perceptron Node

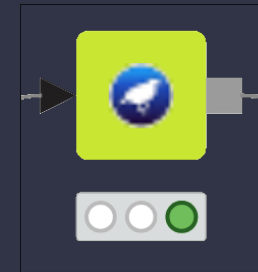
The *MultilayerPerceptron* node is a Weka implementation of MLP.

Similarly to Logistic Regression and SVM, also for MLP **normalization** is required. The MultilayerPerceptron node can perform the normalization setting the corresponding flag in the configuration panel.

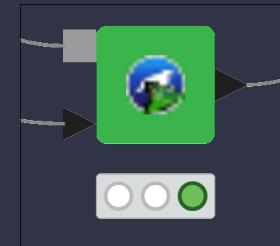
The most relevant parameters are:

- **hiddenLayer**: a sequence of comma separated integer representing the number of nodes for each hidden layer.
- **learningRate**: hyperparameter that determine the step size at each iteration of the optimization algorithm.
-

MultilayerPerceptron



Weka Predictor



Multi Layer Perceptron Node

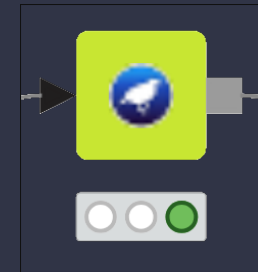
The *MultilayerPerceptron* node is a Weka implementation of MLP.

Similarly to Logistic Regression and SVM, also for MLP **normalization** is required. The MultilayerPerceptron node can perform the normalization setting the corresponding flag in the configuration panel.

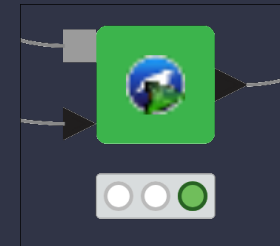
The most relevant parameters are:

- **hiddenLayer**: a sequence of comma separated integer representing the number of nodes for each hidden layer.
- **learningRate**: hyperparameter that determine the step size at each iteration of the optimization algorithm.
- **trainingTime**: number of steps performed by the optimization algorithm.

MultilayerPerceptron



Weka Predictor



Hands on (15 minutes)

- **Extend** the previous workflow adding the **MultiLayer Perceptron** node.
- Test different **architecture**
- Explore the parameters:
 - hiddenLayers
 - learningRate
 - trainingTime
- How does these parameters affect **accuracy**?

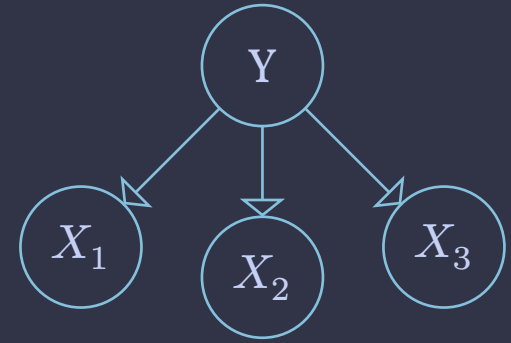


Bayesian Network

Naive Bayes

The *Naive Bayes Classifier* (NBC) is a probabilistic model which assumes that all the features are conditionally independent given the target class.

$$P(X_i | X_j, Y) = P(X_i | Y) \quad \forall i \neq j$$

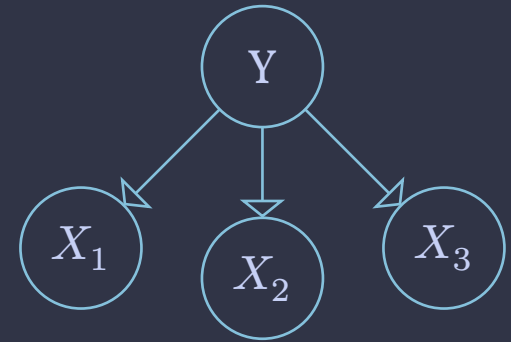


Naive Bayes

The *Naive Bayes Classifier* (NBC) is a probabilistic model which assumes that all the features are conditionally independent given the target class.

$$P(X_i|X_j, Y) = P(X_i|Y) \quad \forall i \neq j$$

Despite the strong assumption of conditional independence, NBC work reasonably well in many complex real-world situations.



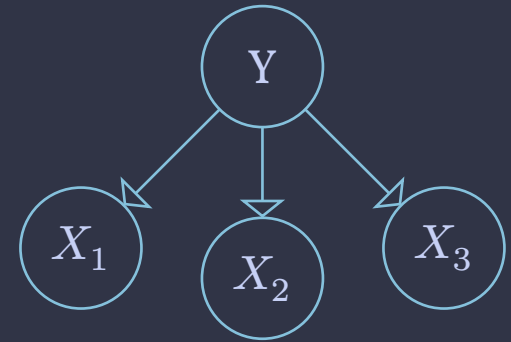
Naive Bayes

The *Naive Bayes Classifier* (NBC) is a probabilistic model which assumes that all the features are conditionally independent given the target class.

$$P(X_i | X_j, Y) = P(X_i | Y) \quad \forall i \neq j$$

Despite the strong assumption of conditional independence, NBC work reasonably well in many complex real-world situations.

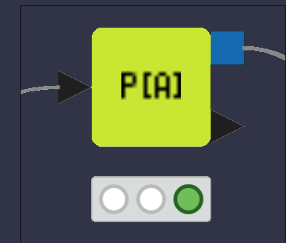
Furthermore, the NBC is parsimonious in both execution time and memory usage.



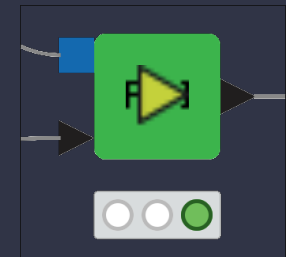
Naive Bayes Node

Naive Bayes Learner and *Naive Bayes Predictor* are nodes provided by the core library of Knime.

Naive Bayes Learner



Naive Bayes Predictor

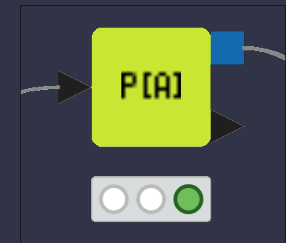


Naive Bayes Node

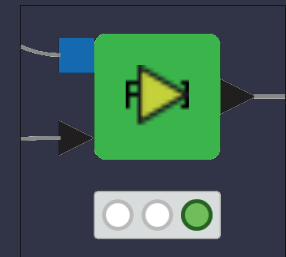
Naive Bayes Learner and *Naive Bayes Predictor* are nodes provided by the core library of Knime.

The simplicity of the NBC is also reflected in the limited number of configurable parameters.

Naive Bayes Learner



Naive Bayes Predictor



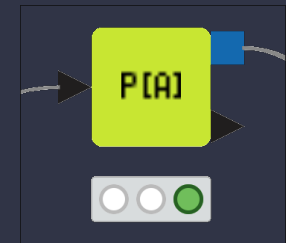
Naive Bayes Node

Naive Bayes Learner and *Naive Bayes Predictor* are nodes provided by the core library of Knime.

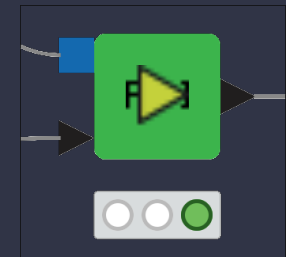
The simplicity of the NBC is also reflected in the limited number of configurable parameters.

In fact, we can simply select the target variable and ignore all the other parameters.

Naive Bayes Learner



Naive Bayes Predictor



Naive Bayes Node

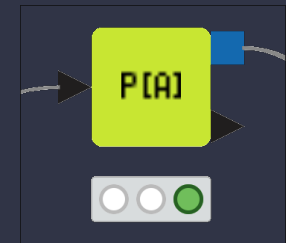
Naive Bayes Learner and *Naive Bayes Predictor* are nodes provided by the core library of Knime.

The simplicity of the NBC is also reflected in the limited number of configurable parameters.

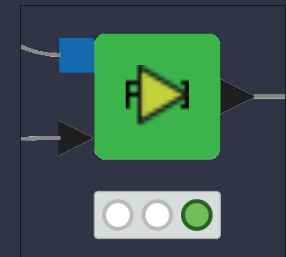
In fact, we can simply select the target variable and ignore all the other parameters.

The parameters *default probability*, *minimum standard deviation* and *threshold standard deviation* are constants used to prevent division by zero during the learning phase.

Naive Bayes Learner



Naive Bayes Predictor



Naive Bayes Node

Naive Bayes Learner and *Naive Bayes Predictor* are nodes provided by the core library of Knime.

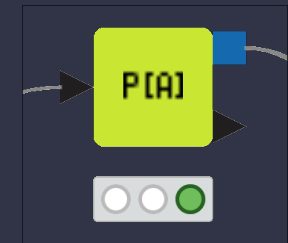
The simplicity of the NBC is also reflected in the limited number of configurable parameters.

In fact, we can simply select the target variable and ignore all the other parameters.

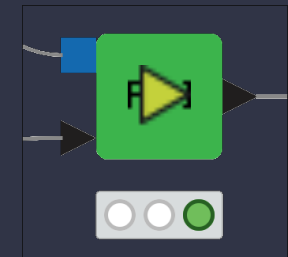
The parameters *default probability*, *minimum standard deviation* and *threshold standard deviation* are constants used to prevent division by zero during the learning phase.

The parameter *Maximum number of unique nominal values per attribute* limits the number of categorical value considered. All nominal columns with more unique values than the defined number will be skipped during learning.

Naive Bayes Learner

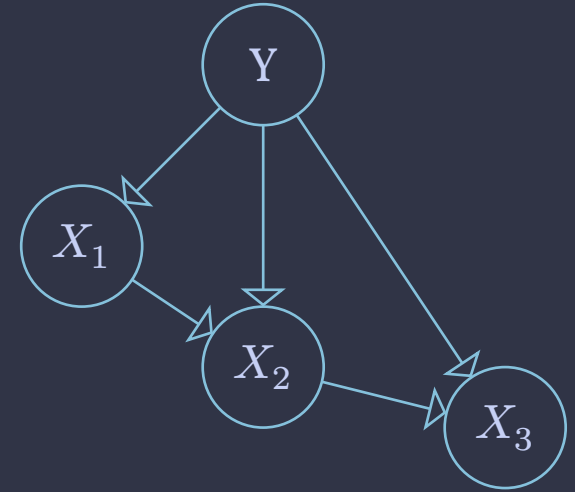


Naive Bayes Predictor



Tree Augmented Naive Bayes

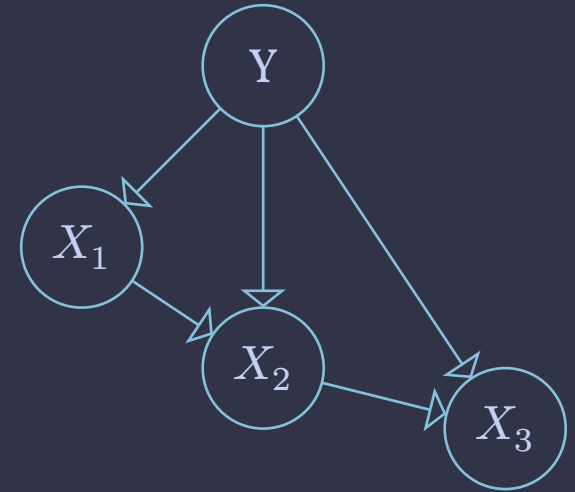
Tree Augmented Naive Bayes Classifier (TANBC) is an extension of the NBC.



Tree Augmented Naive Bayes

Tree Augmented Naive Bayes Classifier (TANBC) is an extension of the NBC.

TANBC relaxes the independence assumption of NBC allowing each feature X_i to depend on the class variable and **at most one other feature**.

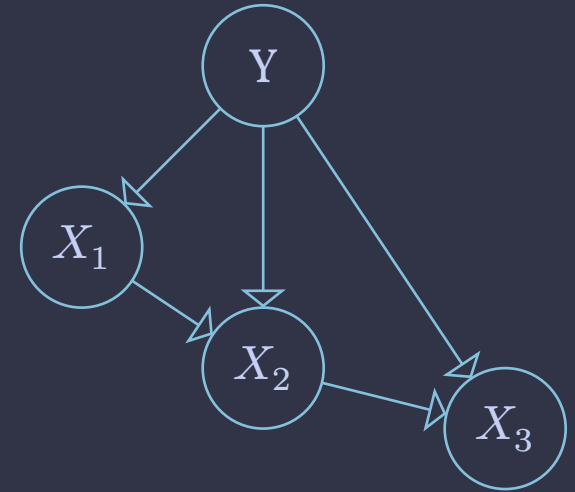


Tree Augmented Naive Bayes

Tree Augmented Naive Bayes Classifier (TANBC) is an extension of the NBC.

TANBC relaxes the independence assumption of NBC allowing each feature X_i to depend on the class variable and **at most one other feature**.

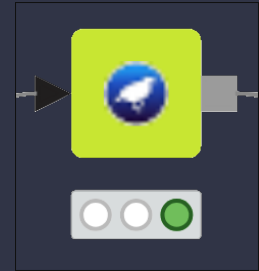
TANBC is a compromise between simplistic NB and a complete Bayesian Network.



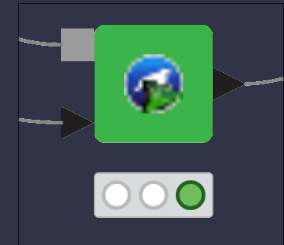
Tree Augmented Naive Bayes Node

The *BayesNet* node provided by Weka, implements a set of algorithm for the training of Bayesian Networks and TANBC.

BayesNet



Weka Predictor

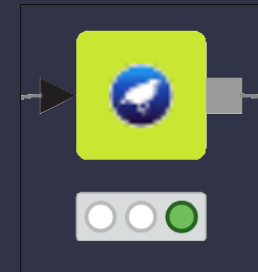


Tree Augmented Naive Bayes Node

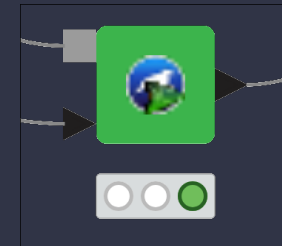
The *BayesNet* node provided by Weka, implements a set of algorithm for the training of Bayesian Networks and TANBC.

The BayesNet node only deals with **discrete variables**. Thus, before using this classifier, we need to utilize a *Binner* node to discretize the numeric variables.

BayesNet



Weka Predictor

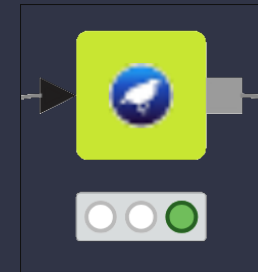


Tree Augmented Naive Bayes Node

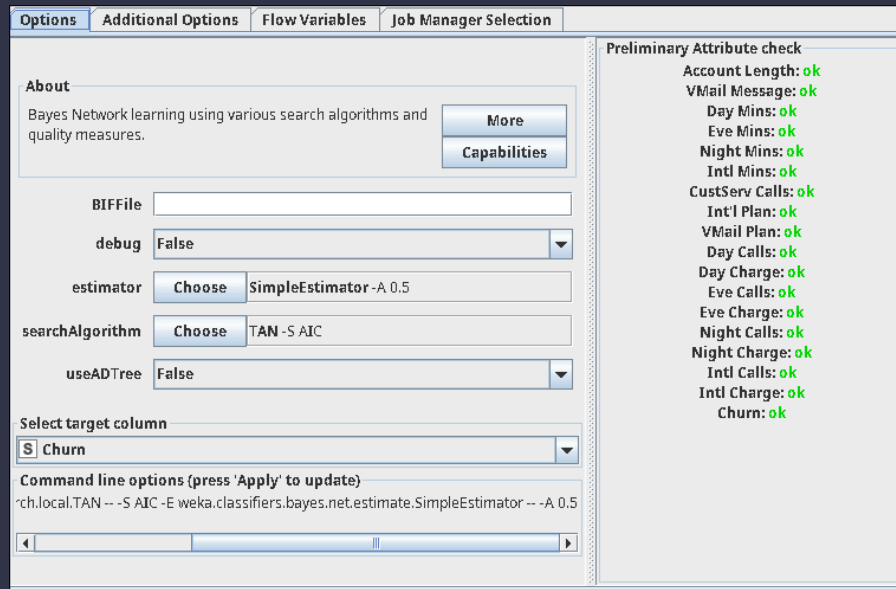
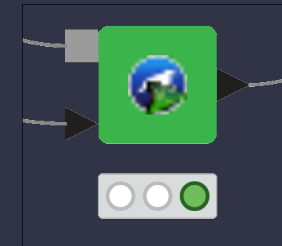
The *BayesNet* node provided by Weka, implements a set of algorithm for the training of Bayesian Networks and TANBC.

The BayesNet node only deals with **discrete variables**. Thus, before using this classifier, we need to utilize a *Binner* node to discretize the numeric variables.

BayesNet



Weka Predictor



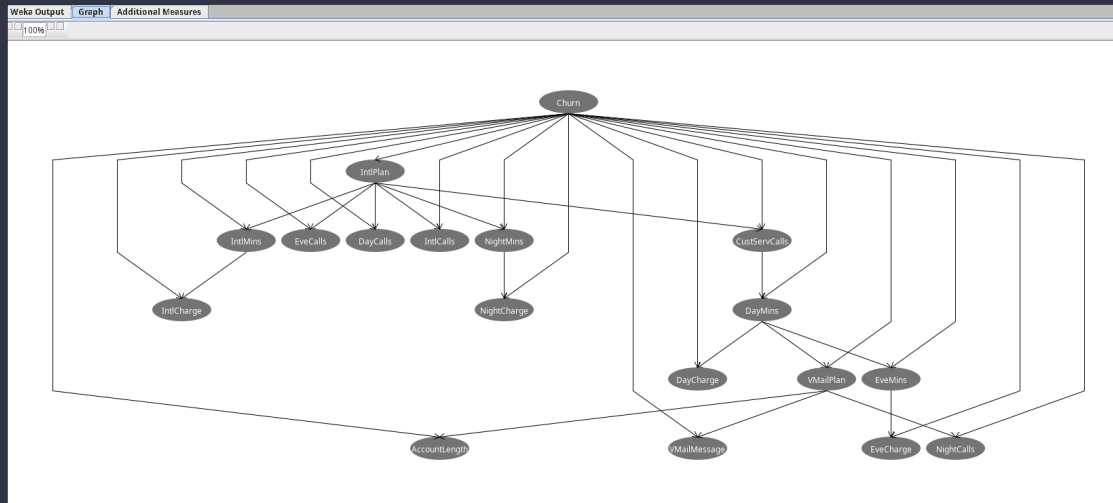
In order to use a TANBC we only need to select the TAN search algorithm

Tree Augmented Naive Bayes Node

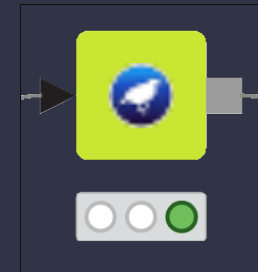
The *BayesNet* node provided by Weka, implements a set of algorithm for the training of Bayesian Networks and TANBC.

The BayesNet node only deals with **discrete variables**. Thus, before using this classifier, we need to utilize a *Binner* node to discretize the numeric variables.

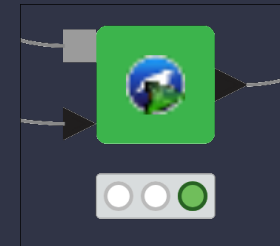
View - Graph



BayesNet

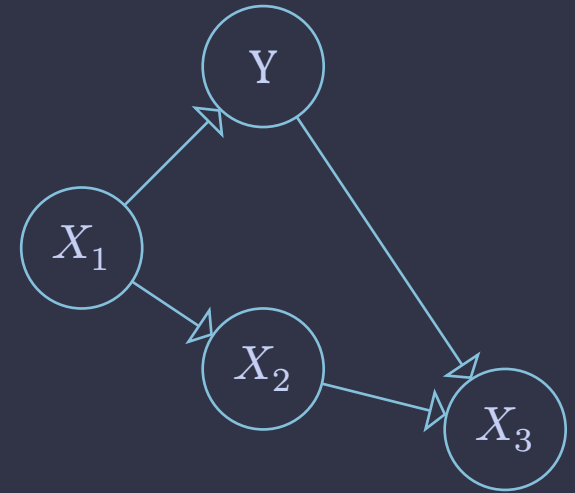


Weka Predictor



Bayesian Network

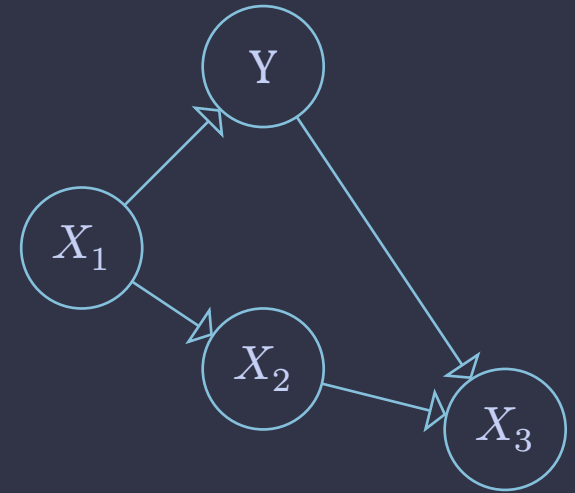
Bayesian Network (BN) is a generalization of TANBC.



Bayesian Network

Bayesian Network (BN) is a generalization of TANBC.

The graph representing a BN is **Directed Acyclic Graph** (DAG).

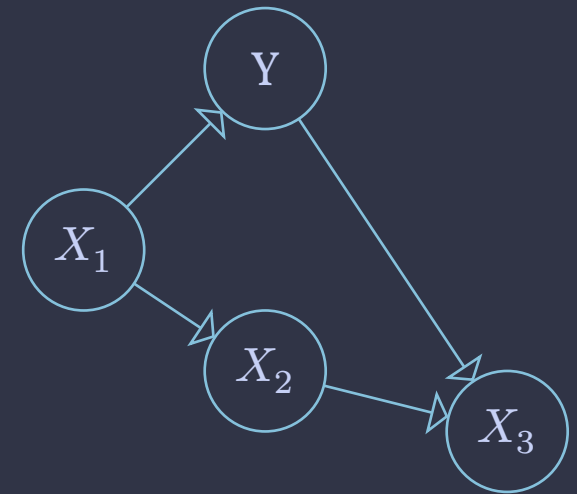


Bayesian Network

Bayesian Network (BN) is a generalization of TANBC.

The graph representing a BN is **Directed Acyclic Graph** (DAG).

Except for acyclicity, no constraint are imposed to the structure of the DAG. Thus, the target variable can have parent nodes and children nodes.



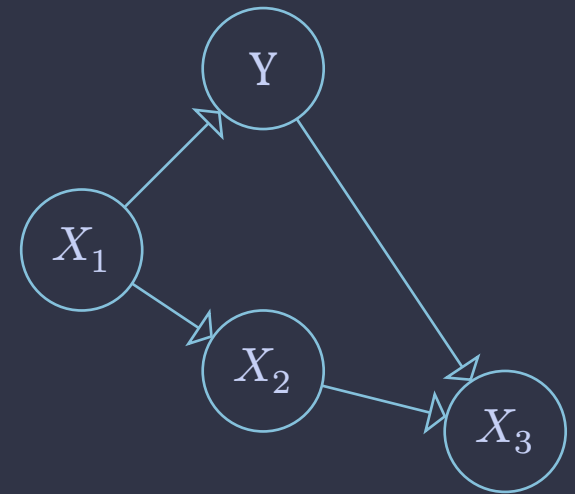
Bayesian Network

Bayesian Network (BN) is a generalization of TANBC.

The graph representing a BN is **Directed Acyclic Graph** (DAG).

Except for acyclicity, no constraint are imposed to the structure of the DAG. Thus, the target variable can have parent nodes and children nodes.

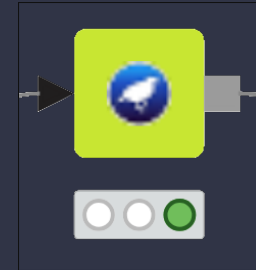
The relaxation of constraint from NBC and TANBC come at the cost of an higher computational complexity and many more hyperparameters to configure.



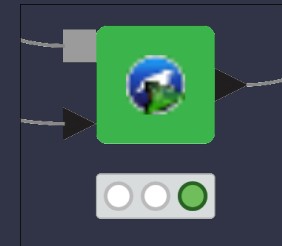
Bayesian Network Node

BayesNet node from Weka library, provide many different algorithms for learning a bayesian network.

BayesNet



Weka Predictor

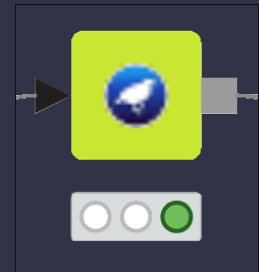


Bayesian Network Node

BayesNet node from Weka library, provide many different algorithms for learning a bayesian network.

The BayesNet node only deals with **discrete variables**. Thus, before using this classifier, we need to utilize a *Binner* node to discretize the numeric variables.

BayesNet



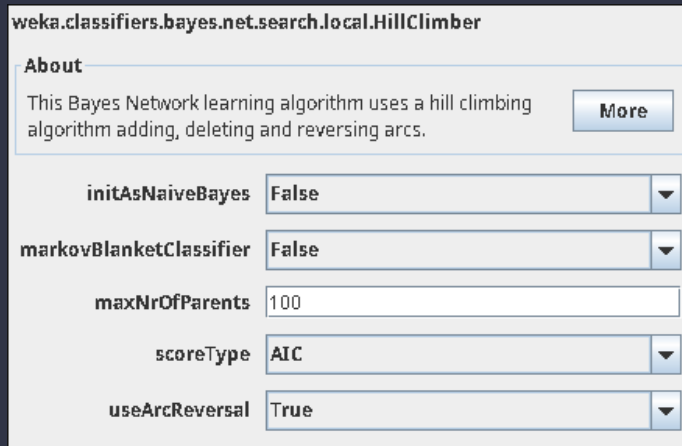
Weka Predictor



Bayesian Network Node

BayesNet node from Weka library, provide many different algorithms for learning a bayesian network.

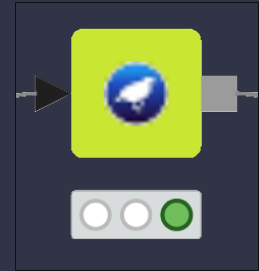
The BayesNet node only deals with **discrete variables**. Thus, before using this classifier, we need to utilize a *Binner* node to discretize the numeric variables.



Similar to TANBC, also for a BN we need to select a **search algorithm**.

The most simple algorithm for BN is **Hill Climbing**.

BayesNet



Weka Predictor

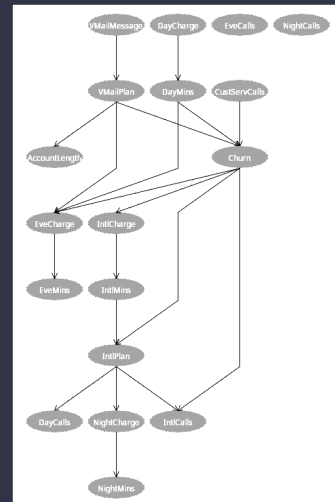


Bayesian Network Node

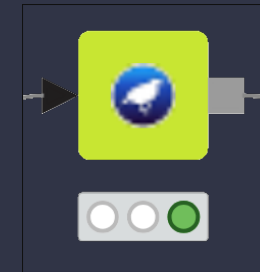
BayesNet node from Weka library, provide many different algorithms for learning a bayesian network.

The BayesNet node only deals with **discrete variables**. Thus, before using this classifier, we need to utilize a *Binner* node to discretize the numeric variables.

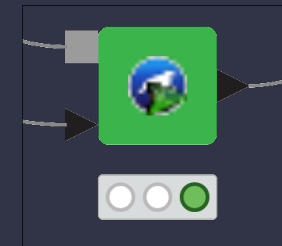
View - Graph



BayesNet



Weka Predictor



Hands on (30 minutes)

- **Import** the **Churn_yn**.
- **Filter** the columns: State, Area Code and Phone
- **Split** the dataset in two partition (80% train, 20% test)
- **Train** a **Naive Bayes Learner**:
 - **Without** any **preprocess**
 - **Discretizing** the variables with the **Binner** node
- How do performances change?
- **Train** a TANBC and a BN with the **BayesNet** node.
 - *Hint: remember to discretize the numerical variables*
- Does discretization affect the performance?
- Does discretization affect the learned structure?

